

x402 Multi-Rail Integration Reference (June 2026): EVM · Algorand (Parsec) · Arweave/Permaweb

TL;DR

- x402 is at **v2** (shipped December 11, 2025, per the official launch post "Introducing x402 V2," authored by Erik Reppel, Carson Roscoe, and Josh Nickerson). Governance moved from a Coinbase+Cloudflare foundation (co-founded September 2025) to the **Linux Foundation in April 2026**, backed by Circle, Google, Microsoft, Stripe, and Visa. The canonical repo is `github.com/x402-foundation/x402` with `coinbase/x402` a development fork. v2 uses **CAIP-2 network IDs**, header-based payment data (`(PAYMENT-REQUIRED)/(PAYMENT-SIGNATURE)/(PAYMENT-RESPONSE)`), modular `@x402/*` SDKs, and is backward-compatible with v1 (`(X-PAYMENT)`, body-based requirements). All three of your rails map cleanly onto the same `(scheme)+(network)` switch. `(GitHub)`
- **EVM** uses the `(exact)` scheme via EIP-3009 `(transferWithAuthorization)` (EIP-712 signed); **Algorand** uses GoPlausible's `(exact)` AVM scheme ("Parsec", `@x402-avm/*`) / PyPI `(x402-avm)` with USDC ASA `(31566704)`, atomic transaction groups + fee abstraction, verified/settled via `algod simulate/submit`; **Arweave** is gated as a fulfillment-desk pattern — accept x402 (USDC/AR) then upload via Turbo SDK.
- **Caveat on PHP:** no public package literally named `(x402-php)` is discoverable on Packagist/GitHub; treat it as your own branded component built on upstream PHP crypto primitives (`(sleepfinance/eip712)`, `(simplito/elliptic-php)`, `(phpseclib)`). The only public PHP x402 code is the brand-new, unproven `(johnguoy/laravel-x402)` (EVM-only, 0 installs). `(Packagist)`

Key Findings

Protocol state (June 2026)

- x402 launched May 2025; per the official x402.org v2 post, "it has processed over 100M payments across APIs, apps, and AI agents," and Chainalysis (via Crypto Briefing) confirms "more than 100 million transactions on Base within nine months of launch." The official x402.org live dashboard shows trailing-30-day figures of **75.41M transactions, \$24.24M volume, 94.06K buyers, and 22K sellers**; CryptoSlate's December 18, 2025 explainer states "By December, it had processed 75 million transactions worth \$24 million."
- v2 changes: CAIP-2 identifiers (`(eip155:8453)`, `(algorand:WGHE2...)`), standards-compliant headers replacing `(X-*)`, modular SDKs under the `@x402` npm org, the **Bazaar** discovery extension, **Sign-In-With-X (SIWx / CAIP-122)** sessions, dynamic `(payTo)` routing, and pluggable multi-facilitator support.

- **Schemes:** `exact` (fixed amount, the shipping scheme), `upto` (usage-capped), and EVM `batch-settlement` (escrow + off-chain vouchers). [GitHub](#)
- **EVM asset transfer methods** within `exact`: EIP-3009 (preferred, gasless), Permit2 (universal ERC-20 fallback via canonical `x402ExactPermit2Proxy` at `0x402085c248EeA27D92E8b30b2C58ed07f9E20001`), and ERC-7710 (smart-account delegation). [GitHub](#)
- **CDP facilitator** offers fee-free USDC settlement on Base and Solana (confirmed by CryptoSlate: "Coinbase's hosted facilitator offers fee-free USDC payments on Base and Solana with high-throughput settlement"); <https://x402.org/facilitator> is the default reference facilitator. [QuickNode](#)

Algorand / Parsec

- Built by **GoPlausible + Algorand Foundation**, merged into the Coinbase repo via **PR #361**; canonical spec is `specs/schemes/exact/scheme_exact_algo.md`.
- Mainnet CAIP-2 = `algorand:wgHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkIt8=`; USDC mainnet ASA = `31566704` (6 decimals). Packages: npm `@x402-avm/core`, `@x402-avm/avm` (+ `express/hono/next/fetch/axios/paywall/extensions`); PyPI `x402-avm` (imports as `from x402...`).
- Payload uses `paymentGroup` (array of base64-msgpack txns, ≤ 16) and `paymentIndex`; fee abstraction puts an unsigned facilitator `pay` txn at index 0 and the client-signed `axfer` at index 1. Verification simulates via `algod /simulate`; settlement submits via `v2/transactions` with instant finality. [GitHub](#)

Arweave / permaweb

- **Turbo SDK** (`@ardrive/turbo-sdk`) by ArDrive/AR.IO bundles ANS-104 data items and accepts top-ups in fiat or crypto (AR, ETH, SOL, ARIO, **base-usdc**). AR.IO already exposes **x402 for USDC on Base** for gateway "fast lane" access, and Turbo supports **just-in-time (JIT)** funding at upload time. [GitHub](#)
- Prior art: **Load Network's "402104"** gates ANS-104 DataItems behind x402 USDC payments. Your fulfillment-desk pattern: gate an upload endpoint behind x402, then call `turbo.upload()`. [Load](#)

Details

1. The x402 v1/v2 HTTP flow

Canonical references:

- Spec: <https://github.com/coinbase/x402/blob/main/specs/x402-specification.md>

- Repo (Foundation): <https://github.com/x402-foundation/x402>; fork: <https://github.com/coinbase/x402>
- Docs: <https://docs.x402.org/core-concepts/facilitator>; v2 launch note: <https://www.x402.org/writing/x402-v2-launch>
- Whitepaper: <https://www.x402.org/x402-whitepaper.pdf>
- HTTP 402: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/402>

Flow:

1. Client `(GET)`s a resource with no payment.
2. Server replies **402 Payment Required** with `(PaymentRequirements)`. In **v1** this is a JSON body `{ x402Version:1, accepts:[...], error }`; in **v2** it is the base64 `(PAYMENT-REQUIRED)` header. `(Circle)`
3. Client picks an entry from `(accepts[])`, builds a `(PaymentPayload)` for that `(scheme)+(network)`, and retries with the payment `(X-PAYMENT)` in v1, `(PAYMENT-SIGNATURE)` in v2). `(GitHub)` `(Blockedden)`
4. Server verifies locally or via facilitator `(POST /verify)`. `(GitHub)`
5. Server settles directly or via facilitator `(POST /settle)`; on success returns **200** with `(X-PAYMENT-RESPONSE)` (v1) / `(PAYMENT-RESPONSE)` (v2) carrying the settlement (base64 JSON, includes tx hash/ID). `(GitHub)` `(GitHub)`

v2 `(PaymentRequirements)` (EVM example):

```
json
{
  "x402Version": 2,
  "accepts": [{
    "scheme": "exact",
    "network": "eip155:8453",
    "amount": "10000",
    "asset": "0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913",
    "payTo": "0xYourReceivingAddress",
    "maxTimeoutSeconds": 60,
    "extra": { "assetTransferMethod": "eip3009", "name": "USDC", "version": "2" }
  }]
}
```

2. EVM integration (node:http, WISDOM-daemon style)

EIP/standard references:

- EIP-3009: <https://eips.ethereum.org/EIPS/eip-3009>

- EIP-712: <https://eips.ethereum.org/EIPS/eip-712>
- EIP-2612: <https://eips.ethereum.org/EIPS/eip-2612>
- ERC-7710: <https://eips.ethereum.org/EIPS/eip-7710>
- EVM exact scheme:
https://github.com/coinbase/x402/blob/main/specs/schemes/exact/scheme_exact_evm.md

Resource server (pure `node:http`, content-addressed, token-bucket, JSON ledger):

ts

```

// evm-resource-server.ts - pure node:http, no framework
import http from "node:http";
import crypto from "node:crypto";
import fs from "node:fs";

const FACILITATOR = process.env.FACILITATOR_URL ?? "https://x402.org/facilitator";
const PAY_TO = process.env.PAY_TO!; // 0x...
const ASSET = "0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913"; // USDC on Base
const NETWORK = "eip155:8453";
const LEDGER = "./ledger.json";

const seen = new Set<string>(JSON.parse(fs.existsSync(LEDGER) ? fs.readFileSync(LEDGER) : "[]"));
const persist = () => fs.writeFileSync(LEDGER, JSON.stringify([...seen]));
const store = new Map<string, Buffer>(); // sha256 -> bytes
const put = (b: Buffer) => { const h = crypto.createHash("sha256").update(b).digest("hex"); store.set(h, b); persist(); };

// token bucket
const bucket = { tokens: 20, last: Date.now() };
function allow() {
    const now = Date.now();
    bucket.tokens = Math.min(20, bucket.tokens + (now - bucket.last) / 1000 * 5);
    bucket.last = now;
    if (bucket.tokens < 1) return false;
    bucket.tokens -= 1; return true;
}

const requirements = () => ({
    x402Version: 1,
    accepts: [{
        scheme: "exact", network: NETWORK,
        maxAmountRequired: "10000", // 0.01 USDC (6 decimals)
        asset: ASSET, payTo: PAY_TO,
        resource: "/data", description: "WISDOM data feed",
        mimeType: "application/json", maxTimeoutSeconds: 60,
        extra: { name: "USDC", version: "2" }
    }],
    error: "X-PAYMENT header is required"
});

async function facilitator(path: string, body: unknown) {
    const r = await fetch(`${FACILITATOR}${path}`, {
        method: "POST", headers: { "content-type": "application/json" },
        body: JSON.stringify(body)
    });
}

```

```

    });
    return r.json();
  }

  http.createServer(async (req, res) => {
    if (!allow()) { res.writeHead(429).end("rate limited"); return; }
    if (req.url !== "/data") { res.writeHead(404).end(); return; }

    const xpay = req.headers["x-payment"] as string | undefined;
    if (!xpay) { res.writeHead(402, {"content-type": "application/json"}).end(JSON.stringify({ error: "no payment" })); return; }

    const payload = JSON.parse(Buffer.from(xpay, "base64").toString("utf8"));
    const reqs = requirements().accepts[0];

    // replay protection on the EIP-3009 nonce
    const nonce = payload?.payload?.authorization?.nonce;
    if (nonce && seen.has(nonce)) { res.writeHead(409).end("replay"); return; }

    const verify = await facilitator("/verify", { x402Version: 1, paymentPayload: payload });
    if (!verify.isValid) { res.writeHead(402, {"content-type": "application/json"}).end(JSON.stringify({ error: "invalid verification" })); return; }

    const settle = await facilitator("/settle", { x402Version: 1, paymentPayload: payload });
    if (!settle.success) { res.writeHead(402).end(JSON.stringify({ error: settle.error })); return; }
    if (nonce) { seen.add(nonce); persist(); }

    const body = Buffer.from(JSON.stringify({ wisdom: "42", ts: Date.now() }));
    const cid = put(body);
    res.writeHead(200, {
      "content-type": "application/json",
      "x-content-sha256": cid,
      "x-payment-response": Buffer.from(JSON.stringify(settle)).toString("base64")
    }).end(body);
  }).listen(8402);

```

Client signing (EIP-712 / EIP-3009 with viem):

ts

```

import { createWalletClient, http as vhttp } from "viem";
import { privateKeyToAccount } from "viem/accounts";
import { base } from "viem/chains";

const account = privateKeyToAccount(process.env.PK as `0x${string}`);
const wallet = createWalletClient({ account, chain: base, transport: vhttp() });

const auth = {
  from: account.address,
  to: "0xYourReceivingAddress",
  value: 10000n, // 0.01 USDC
  validAfter: 0n,
  validBefore: BigInt(Math.floor(Date.now()/1000) + 300),
  nonce: `0x${crypto.randomBytes(32).toString("hex")}` as `0x${string}`
};

const signature = await wallet.signTypedData({
  domain: { name: "USD Coin", version: "2", chainId: 8453, verifyingContract: "0x8335"
  types: { TransferWithAuthorization: [
    {name:"from",type:"address"},{name:"to",type:"address"},{name:"value",type:"uint256"},
    {name:"validAfter",type:"uint256"},{name:"validBefore",type:"uint256"},{name:"nonce",type:"uint256"}
  ]},
  primaryType: "TransferWithAuthorization",
  message: auth
});

const header = Buffer.from(JSON.stringify({
  x402Version: 1, scheme: "exact", network: "eip155:8453",
  payload: { signature, authorization: {
    from: auth.from, to: auth.to, value: auth.value.toString(),
    validAfter: auth.validAfter.toString(), validBefore: auth.validBefore.toString(),
  }
  })).toString("base64");
// retry GET /data with header "X-PAYMENT": header

```

Foundry test scaffolding (for any on-chain settlement/forwarder contract; mainnet-only deploy targets):

solidity

```
// test/Settlement.t.sol - forge test
// Foundry book: https://book.getfoundry.sh
pragma solidity ^0.8.24;
import "forge-std/Test.sol";

interface IERC3009 {
    function transferWithAuthorization(address from,address to,uint256 value,
        uint256 validAfter,uint256 validBefore,bytes32 nonce,uint8 v,bytes32 r,bytes32 s)
}

contract SettlementTest is Test {
    IERC3009 usdc = IERC3009(0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913); // Base mainnet
    function setUp() public { vm.createSelectFork(vm.envString("BASE_RPC_URL")); }

    function test_transferWithAuthorization() public {
        uint256 pk = 0xA11CE;
        address from = vm.addr(pk);
        bytes32 nonce = keccak256("x402-nonce-1");
        // ... build EIP-712 digest, vm.sign(pk, digest) -> (v,r,s), then call usdc.trans
        // assert recipient balance increased and a replayed nonce reverts.
    }
}
```

The EVM exact spec also defines the **Permit2** path (`permitWitnessTransferFrom`) via the canonical `x402ExactPermit2Proxy`, CREATE2-deployed at `0x402085c248EeA27D92E8b30b2C58ed07f9E20001`) and **ERC-7710** delegation (verified entirely through simulation of `redeemDelegations`). Use EIP-3009 for USDC/EURC; fall back to Permit2 for arbitrary ERC-20s.

3. Algorand integration — GoPlausible "Parsec"

References:

- Algorand exact scheme:
https://github.com/coinbase/x402/blob/main/specs/schemes/exact/scheme_exact_algo.md
- PR #361: <https://github.com/coinbase/x402/pull/361>
- GoPlausible docs: <https://github.com/GoPlausible/.github/blob/main/profile/algorand-x402-documentation/README.md>
- Landing/facilitator: <https://x402.goplausible.xyz/> and <https://facilitator.goplausible.xyz/>
- Algorand indexer: <https://dev.algorand.co/reference/rest-api/indexer/operations/searchfortransactions/>; algod simulate/submit:
<https://developer.algorand.org/docs/rest-apis/algod/>

The **exact** AVM scheme transfers USDC ASA 31566704 via an **atomic transaction group**. The **payload** contains **paymentGroup** (array of base64-msgpack txns, ≤16) and **paymentIndex**. With fee abstraction, index 0 is an **unsigned facilitator** **pay** txn (pooled fee, note x402-fee-payer) and index 1 is the **client-signed** **axfer** (note x402-payment-v2). The client holds no ALGO for fees.

Per the canonical spec, verification (1) validates **x402Version**=2, **scheme**="exact", and **network** match; (2) checks **paymentGroup** ≤ 16 elements; (3) decodes all txns; (4) at **paymentIndex** checks **aamt**==amount, **arcv**==payTo, **xaid**==asset; (5) for any txn whose **snd** is the facilitator address, verifies it is a **pay** with **close**/**rekey**/**amt** omitted and a reasonable fee, then signs it; (6) simulates the whole group against algod (**/simulate**). Settlement submits the verified group to algod (**v2/transactions**) (instant finality, no consensus forks). GoPlausible's SDK enforces **MAX_REASONABLE_FEE = 10,000,000** microAlgos and rejects **keyreg**, unbalanced **rekeyTo**, and **closeRemainderTo**/**assetCloseTo** (anti-drain).

v2 PAYMENT-SIGNATURE payload (mainnet, fee-abstracted):

```
json
{
  "x402Version": 2, "scheme": "exact",
  "network": "algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=",
  "accepted": {
    "scheme": "exact",
    "network": "algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=",
    "amount": "5000000", "asset": "31566704",
    "payTo": "RESOURCESEVERADDRESS...LTSRPAE", "maxTimeoutSeconds": 60,
    "extra": { "feePayer": "FACILITATORADDRESS...LQCXBZE" }
  },
  "payload": {
    "paymentIndex": 1,
    "paymentGroup": ["gaN0eG6Jo2ZlZc0H0KJm4DLgNro2dlbqxtYwLubmV0...", "gqNzaWfEQP3..."]
  }
}
```

The settlement response returns the txid of **paymentGroup[paymentIndex]**:

```
json
{
  "success": true, "errorReason": null, "payer": "<payer>",
  "transaction": "NTRZR6HGMMZGYMJKUNVNLKLA427ACAVIPFNC6JHA5XNBQHW7MWA",
  "network": "algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8="
}
```

WISDOM-daemon-style facilitator verifying live against Algorand (node:http). Per the spec, verification simulates the group against an algod node and settlement submits via `/v2/transactions`. You can additionally cross-check the settled txn against the **indexer** (`/v2/assets/31566704/transactions` or `/v2/transactions/{txid}`) for the JSON ledger and replay protection: [GitHub](#)

ts

```
// avm-facilitator.ts - pure node:http; verify via algod /simulate, settle via /v2/tr
// confirm via indexer. Uses @algorandfoundation/algokit-utils internally (algosdk no
import http from "node:http";

const ALGOD = "https://mainnet-api.algonode.cloud";
const INDEXER = "https://mainnet-idx.algonode.cloud";
const USDC_ASA = 31566704;

async function indexerConfirm(txid: string) {
  const r = await fetch(`${INDEXER}/v2/transactions/${txid}`);
  if (!r.ok) return null;
  const j = await r.json();
  const t = j.transaction;
  return t?["asset-transfer-transaction"]?["asset-id"] === USDC_ASA ? t : null;
}

http.createServer((req, res) => {
  // POST /verify -> decode paymentGroup, check aamt/arcv/xaid at paymentIndex,
  //               sign facilitator pay txn, simulate group via ${ALGOD}/v2/transa
  // POST /settle -> submit signed group to ${ALGOD}/v2/transactions, wait for confi
  //               then indexerConfirm(txid) and append {txid, payer, round} to JS
  res.writeHead(200, {"content-type": "application/json"}).end(JSON.stringify({ ok: tr
})).listen(4000);
```

Express-equivalent registration (the supported high-level path) for reference:

ts

```

import express from "express";
import { paymentMiddlewareFromConfig } from "@x402-avm/express";
import { registerExactAvmScheme } from "@x402-avm/avm/exact/server";
import { x402ResourceServer, HTTPFacilitatorClient } from "@x402-avm/core/server";
import { ALGORAND_MAINNET_CAIP2, USDC_MAINNET_ASA_ID } from "@x402-avm/avm";

const app = express();
const server = new x402ResourceServer(new HTTPFacilitatorClient({ url: "https://facil
registerExactAvmScheme(server);
app.use(paymentMiddlewareFromConfig({
  "GET /api/premium": {
    accepts: { scheme: "exact", network: ALGORAND_MAINNET_CAIP2,
      payTo: process.env.ALGO_PAYTO!, price: { asset: USDC_MAINNET_ASA_ID, amount: "1
    description: "Premium API"
  }
}, server));

```

Python (mindX BDI API side), framework-agnostic:

```

python

from x402 import x402ResourceServer
from x402.mechanisms.avm.exact import register_exact_avm_server
from x402.mechanisms.avm.constants import ALGORAND_MAINNET_CAIP2, USDC_MAINNET_ASA_ID
server = x402ResourceServer(facilitator_url="https://facilitator.goplausible.xyz")
register_exact_avm_server(server) # pip install "x402-avm[avm]"; import as x402

```

ARC context: USDC on Algorand is an **ASA** (the original Algorand Standard Asset), not an ARC-200 smart-contract token; the **exact** AVM scheme settles ASA **axfer** transactions. ARC-200 (app-based tokens) is **not** required for the canonical USDC path. Client signers conform to the **ClientAvmSigner** interface and are compatible with **@txnlab/use-wallet** (Pera, Defly, Lute).

[github](#)

4. Arweave / permaweb fulfillment desk

References:

- Turbo SDK: <https://github.com/ardriveapp/turbo-sdk>; docs <https://docs.ardrive.io/docs/turbo/turbo-sdk/> and <https://docs.ar.io/sdks/turbo-sdk>
- AR.IO docs: <https://docs.ar.io>; Turbo app <https://turbo.ar.io>
- Arweave HTTP API: <https://docs.arweave.org/developers/arweave-node-server/http-api>

- x402+Arweave prior art (Load 402104): <https://blog.load.network/x402/> and <https://402.load.network/>

Pattern: x402 gates the *payment*; the fulfillment desk holds Turbo Credits (or funds JIT) and performs the permanent upload. THOT/THLNK permaweb writes become a paid endpoint: collect USDC (EVM or AVM) or AR, then `turbo.upload()`. Turbo Credits maintain a 1:1 storage-purchasing-power peg to AR, are non-refundable/non-transferable, and uploads under 100 KiB are free.

ts

```
// arweave-fulfillment.ts — gate upload behind x402, then bundle to Arweave via Turbo
import http from "node:http";
import { TurboFactory, ArweaveSigner } from "@ardrive/turbo-sdk";
import fs from "node:fs";

const jwk = JSON.parse(fs.readFileSync("./key.json", "utf8"));
const turbo = TurboFactory.authenticated({ signer: new ArweaveSigner(jwk) }); // desk
const FACILITATOR = "https://x402.org/facilitator";

http.createServer(async (req, res) => {
  if (req.method !== "POST" || req.url !== "/permaweb/upload") { res.writeHead(404).end(); }
  const xpay = req.headers["x-payment"] as string | undefined;
  const reqs = { scheme: "exact", network: "eip155:8453", maxAmountRequired: "50000", //
    asset: "0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913", payTo: process.env.PAY_TO!,
    resource: "/permaweb/upload", description: "Permanent Arweave storage (THOT/THLNK)",
    mimeType: "application/json", maxTimeoutSeconds: 60, extra: { name: "USDC", version: "2" } };
  if (!xpay) { res.writeHead(402, { "content-type": "application/json" }).end(JSON.stringify({ error: "no payment" })); }

  const payload = JSON.parse(Buffer.from(xpay, "base64").toString());
  const v = await (await fetch(`${FACILITATOR}/verify`, { method: "POST", headers: { "content-type": "application/json" }, body: JSON.stringify(payload) }));
  if (!v.isValid) { res.writeHead(402).end(JSON.stringify({ error: v.invalidReason })); }
  const s = await (await fetch(`${FACILITATOR}/settle`, { method: "POST", headers: { "content-type": "application/json" }, body: JSON.stringify(payload) }));
  if (!s.success) { res.writeHead(402).end(JSON.stringify({ error: s.errorReason })); }

  const chunks: Buffer[] = []; for await (const c of req) chunks.push(c as Buffer);
  const data = Buffer.concat(chunks);
  const { id } = await turbo.upload({
    data,
    dataItemOpts: { tags: [{ name: "Content-Type", value: "application/octet-stream" }], },
  });
  res.writeHead(200, { "content-type": "application/json", "x-payment-response": Buffer.from(JSON.stringify({ arweaveId: id, url: `https://arweave.net/${id}` })).toString() });
}).listen(8403);
```

For a fully native crypto top-up (no fiat desk), Turbo accepts **base-usdc** and AR directly: **Ar**

```
ts

import { TurboFactory, USDCToTokenAmount } from "@ardrive/turbo-sdk";
const t = TurboFactory.authenticated({ signer, token: "base-usdc" });
await t.topUpWithTokens({ tokenAmount: USDCToTokenAmount(1) }); // $1 of credits
```

JIT funding (avoids pre-funding risk — top up exactly at upload):

ts

```
import { OnDemandFunding } from "@ardrive/turbo-sdk";
const fundingMode = new OnDemandFunding({ maxTokenAmount: { "base-usdc": 25 }, topUpFunding: { "base-usdc": 25 } });
await turbo.uploadFile({ file, fundingMode });
```

5. Unified framework-agnostic middleware (CAIP-2 router)

Matches the `allchain.html` chainmapping concept: one switch keyed off CAIP-2 `network`.

ts

```
type Rail = "evm" | "avm" | "arweave";
function railFor(network: string): Rail {
  if (network.startsWith("eip155:")) return "evm";
  if (network.startsWith("algorand:")) return "avm";
  if (network === "arweave:permaweb") return "arweave"; // synthetic id for the fulfillment
  throw new Error(`unsupported network ${network}`);
}

// Each accepts[] entry carries scheme+network; the daemon dispatches verify/settle to the
// matching handler: exact(EVM EIP-3009) | exact(AVM atomic group) | arweave fulfillment
// SIWx (CAIP-122) sessions can short-circuit repeat calls; login333 issues the CAIP-10
// using CAIP-2 chain ids consistent with the registry at agenticplace.pythai.net/allchain
```

A single `accepts[]` can offer multiple rails simultaneously (the GoPlausible paywall does exactly this — e.g. an Algorand `algorand:...` option and an EVM `eip155:84532` option on the same route), letting the client/agent choose its rail. Build your daemon to publish one `accepts` array spanning all three deployments.

- CAIP-2: <https://chainagnostic.org/CAIPs/caip-2>; CAIP-122 (SIWx): <https://chainagnostic.org/CAIPs/caip-122>
- For BANKON's login333 universal identity, CAIP-122 generalizes Sign-In-With-Ethereum across namespaces (its `chainId` segment is a CAIP-2 id; address is the CAIP-10 account segment). x402 v2 ships SIWx as an extension for reusable sessions — letting a wallet skip re-payment on repeat calls, which is the right primitive for mindX's autonomous repeated API access.

6. PHP coverage

There is **no public package literally named** `x402-php`. Build it on upstream PHP primitives:

- EIP-712 hashing: `(sleepfinance/eip712)` — <https://github.com/sleepfinance/eip712>
- secp256k1 + ed25519: `(simplito/elliptic-php)` — <https://packagist.org/packages/simplito/elliptic-php>
- ed25519 / general crypto: `(phpseclib/phpseclib)` — <https://github.com/phpseclib/phpseclib>
- Algorand SDK: `(ffsolutions/php-algorand-sdk)` — <https://github.com/ffsolutions/php-algorand-sdk>
- Nearest public x402 PHP code (EVM-only, unproven, 0 installs): `(johnguoy/laravel-x402)` — <https://github.com/JohnGuoy/laravel-x402>

EVM `(exact)` (EIP-712/EIP-3009) signing in PHP:

```
php

use SleepFinance\Eip712;
use Elliptic\EC;

$typed = [ /* EIP712Domain + TransferWithAuthorization typed data with the auth messa
$eip712 = new Eip712($typed);
$digest = $eip712->hashTypedDataV4();
$ec = new EC('secp256k1');
$key = $ec->keyFromPrivate(getenv('PK'));
$sig = $key->sign($digest, ['canonical' => true]);
$signature = '0x' . $sig->r->toString(16) . $sig->s->toString(16) . dechex($sig->reco
// base64 the {scheme:"exact", network:"eip155:8453", payload:{signature, authorizati
```

Algorand `(exact)` (Ed25519) signing in PHP:

```
php

use Elliptic\EdDSA;
$ec = new EdDSA('ed25519');
$key = $ec->keyFromSecret(getenv('ALGO_SK_HEX')); // Algorand account secret
$sig = $key->sign(bin2hex($txnMsgpackBytes))->toHex(); // sign the axfer txn bytes
// assemble paymentGroup (base64 msgpack txns) + paymentIndex, base64 into X-PAYMENT
```

Note: Algorand canonically uses Ed25519 over the msgpack txn bytes prefixed with `(TX)`; `(php-algorand-sdk)` handles the encoding/prefix, while `(elliptic-php)`/`(phpseclib)` provide the raw Ed25519 primitive. Validate any PHP Ed25519 output against `(algosdk)` before trusting it in production.

7. 2026-current developments

- **Governance:** x402 moved to the **Linux Foundation in April 2026** (per Chainalysis via Crypto Briefing), with Circle, Google, Microsoft, Stripe, and Visa backing — relevant to long-term protocol stability for your ecosystem. `(coinbase/x402)` is now a development fork of `(x402-foundation/x402)`.
- **Facilitators:** CDP (fee-free USDC on Base + Solana), x402.rs (Rust), PayAI (multi-chain), GoPlausible's AVM facilitator, and **Stripe**, which integrated x402 for USDC on Base by February 2026 (preview API version `(2026-03-04)`).
- **Algorand** `(exact)` **scheme merged upstream** (PR #361); GoPlausible v2.6+ dropped `(algosdk)` for `(@algorandfoundation/algokit-utils@10.0.0-alpha.39)`.
- **ERC-8004** (trustless agent identity/reputation) launched on Ethereum mainnet **January 29, 2026** and pairs with x402 — directly relevant to AgenticPlace's agent marketplace.
- **Arweave x402:** AR.IO/Turbo expose x402 USDC-on-Base for gateway fast-lane; Load Network's 402104 gates ANS-104 DataItems (expirable, private, paywalled).
- **Payment-size shift:** Chainalysis reports payments over \$1 grew from ~49% of value transferred in early 2025 to ~95% by early 2026 — x402 is moving beyond sub-cent micropayments toward real commerce.
- **Ambiguity flags:** v1↔v2 header duality (`(X-PAYMENT)` vs `(PAYMENT-SIGNATURE)`); GoPlausible package import quirk (install `(x402-avm)`, import `(x402)`); no audited PHP x402 lib; AVM verification is `algod-(simulate)`-based in spec while GoPlausible's SDK also supports indexer lookups; x402 itself has had no formal third-party security audit per multiple secondary sources.

Recommendations

1. **Adopt v2 + CAIP-2 across all three deployments now**, but keep v1 fallback parsing — the ecosystem is mid-migration and your `(node:http)` daemon should read both `(PAYMENT-SIGNATURE)` (v2) and `(X-PAYMENT)` (v1). Key your unified daemon off CAIP-2 `(network)`, mirroring `(allchain.html)`.
2. **EVM:** use EIP-3009 USDC on Base mainnet via the CDP facilitator first (fee-free, fastest), self-host the `(node:http)` resource server you control, and keep replay protection on the 3009 nonce in your JSON ledger.
3. **Algorand:** standardize on USDC ASA `(31566704)` + fee abstraction so clients need no ALGO; run your WISDOM-daemon facilitator that simulates via `algod (/simulate)`, submits via `(v2/transactions)`, and double-records settled txids from the indexer for the constitutional/BANKON ledger.
4. **Arweave:** run the fulfillment desk holding Turbo Credits; gate uploads behind x402 (USDC) and use JIT funding to avoid pre-funding risk. Watch Load 402104 for expirable/private

DataItem patterns you can mirror in THOT/THLNK.

5. **PHP:** ship your `(x402-php)` as a thin layer over `(sleepfinance/eip712)` + `(simplito/elliptic-php)` (EVM) and `(phpseclib)/(elliptic-php)` ed25519 + `(php-algorand-sdk)` (AVM); do not depend on `(johnguoy/laravel-x402)` in production.
6. **Identity:** wire login333 to CAIP-122/SIWx so a single wallet signature establishes a reusable session across AgenticPlace, mindX, and BANKON — and so mindX's autonomous repeated calls don't re-pay per request.
7. **Thresholds to revisit:** if CDP introduces fees or an AVM facilitator SLA degrades, self-host the facilitator; if v3/new schemes ship or Turbo exposes a first-class x402 upload endpoint, replace the fulfillment-desk shim with the native integration; if a payment-size threshold pushes you above \$1/call, prefer the `(exact)` scheme's strong settlement guarantees over batched/`(upto)` flows.

Caveats

- Per-rail finality and trust models differ (EVM gasless 3009 vs AVM atomic-group simulate vs Turbo Credit bundling); price micro-endpoints with buffers for gas/peg movement, and note x402 has no formal third-party security audit per multiple secondary trackers.
- "Parsec"/x402-avm and "login333"/x402-php are your branded components; the authoritative upstreams are GoPlausible's `(@x402-avm/*)` (npm) and PyPI `(x402-avm)`, plus the PHP crypto primitives above. There is no upstream `(x402-php)` or official x402 PHP SDK to point to.
- Some ecosystem figures (the ~\$800M Q4 2025 ecosystem market cap tracked by CoinGecko/Artemis, transaction counts) come from secondary trackers and dashboards and should be treated as point-in-time approximations rather than audited totals.
- The AVM payload examples use abbreviated/illustrative base64 transaction blobs and placeholder addresses; reconstruct real `(paymentGroup)` entries with `(@x402-avm/avm)` or the Python `(x402)` package, which build and msgpack-encode the atomic group for you.